

EE-170L Computer Programming Lab
Laboratory Session 13

Structures in C++

Name:	Shahzad Gul
Semester:	2
Section:	B
Reg No:	0671
Lab:	13
Date:	Spring 2026

1. Objective

The objective of this lab session is to understand and apply the concepts of structures in C++ programming. Students will learn how to declare structures, initialize structure variables, access structure members using the dot operator, and work with multiple structure variables. By the end of this lab, students will be able to create custom data types using structures to group related data items of different types.

Key learning outcomes include:

- Declaring structures with multiple members of different data types
- Initializing structure variables using the dot operator
- Accessing and displaying structure member values
- Working with multiple variables of the same structure type
- Understanding the difference between structures and arrays

2. Theory

2.1 Structures in C++

A structure is a collection of simple variables that can have the same or different data types (int, float, char, string, etc.). This is unlike an array, in which all the variables must be the same type. The data items in a structure are called the **members** of the structure. Structures allow programmers to group related data under a single name, making the code more organized and readable.

Key characteristics of structures:

- The closing brace in the structure declaration must be followed by a semicolon
- A structure declaration defines the 'form' of the structure but does not reserve memory
- Memory is allocated only when a structure variable is declared
- One structure can be nested within another structure
- Structure variables can be passed to functions (individual members or entire variable)

2.2 Structure Declaration Syntax

The syntax for declaring a structure is as follows. The **struct** keyword is followed by the structure name (tag), and the members are enclosed in braces. The declaration ends with a semicolon.

```
struct StructureName
{
    dataType member1;
    dataType member2;
    dataType member3;
};
```

2.3 Accessing Structure Members

Structure members are accessed using the **dot operator (.)**. The general syntax is: **variableName.memberName**. For example, if **person1** is a variable of type **Person** with a member **name**, we access it as **person1.name**. This allows us to read or assign values to individual members of the structure.

2.4 Structure Variable Initialization

Structure variables can be initialized in several ways: (1) by assigning values to each member using the dot operator after declaration, (2) by using an initialization list at the time of declaration, or (3) by copying one structure variable to another. For example: **part part1 = {6244, 373, 217.55F};** initializes all members in one statement.

Comparison: Structures vs Arrays

Feature	Structure	Array
---------	-----------	-------

Data Types	Members can be of different types	All elements must be the same type
Access Method	Dot operator (.) by member name	Index operator [] by position
Memory	Allocated when variable is declared	Allocated when array is declared
Use Case	Grouping related data of different types	Storing multiple values of the same type

3. Lab Tasks

Task 1: Structure Declaration and Initialization (Person)

Objective: Declare a structure called **Person** with the following members: name (string), age (integer), and address (string). Declare a variable called **person1** of type Person and initialize its members with values of your choice. Display the values of the members of **person1** using the dot operator.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 13 - Task 1: Structure Declaration and Initialization (Person)
 * Description: Declare a Person structure with name, age, and address members.
 *              Initialize and display the values using dot operator.
 */

#include <iostream>
#include <string>      // Required for string class
using namespace std;

// Declare a structure called 'Person' with the required members
struct Person
{
    string name;        // Member: name (string)
    int age;            // Member: age (integer)
    string address;     // Member: address (string)
};

int main()
{
    // Declare a variable called person1 of type Person
    // Initialize its members with values
    Person person1;

    person1.name = "Shahzad Gul";
    person1.age = 20;
    person1.address = "UET Peshawar, KPK, Pakistan";

    // Display the values of the members of person1 using dot operator
    cout << "=== Person 1 Information ===" << endl;
    cout << "Name:      " << person1.name << endl;
    cout << "Age:       " << person1.age << endl;
    cout << "Address:  " << person1.address << endl;

    return 0;
}
```

Expected Output:

```
=== Person 1 Information ===  
Name:    Shahzad Gul  
Age:     20  
Address: UET Peshawar, KPK, Pakistan
```

Explanation: The program declares a **Person** structure with three members: **name** (string), **age** (int), and **address** (string). A variable **person1** of type **Person** is declared and its members are initialized using the dot operator. The program then displays all member values, demonstrating how to group related data of different types under a single structure.

Task 2: Structure Declaration and Initialization (Student)

Objective: Declare a structure called **Student** with the following members: name (string), age (integer), and grade (char). Declare a variable of type **Student** called **student1** and initialize its members with values of your choice. Display values of **student1** structure.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 13 - Task 2: Structure Declaration and Initialization (Student)
 * Description: Declare a Student structure with name, age, and grade members.
 *              Initialize and display the values.
 */

#include <iostream>
#include <string>      // Required for string class
using namespace std;

// Declare a structure called 'Student' with the required members
struct Student
{
    string name;        // Member: name (string)
    int age;            // Member: age (integer)
    char grade;         // Member: grade (char)
};

int main()
{
    // Declare a variable of type Student called student1
    // Initialize its members with values
    Student student1;

    student1.name = "Shahzad Gul";
    student1.age = 20;
    student1.grade = 'A';

    // Display values of student1 structure
    cout << "=== Student 1 Information ===" << endl;
    cout << "Name:  " << student1.name << endl;
    cout << "Age:   " << student1.age << endl;
    cout << "Grade: " << student1.grade << endl;

    return 0;
}
```

Expected Output:

```
=== Student 1 Information ===
Name:  Shahzad Gul
Age:   20
```

Grade: A

Explanation: The program declares a **Student** structure with three members of different types: **name** (string), **age** (int), and **grade** (char). The variable **student1** is initialized and its values are displayed. This task reinforces the concept that structure members can be of any data type, including character types for single-letter grades.

Task 3: Accessing Structure Elements (Book)

Objective: Declare a structure called **Book** with the following members: title (string), author (string), price (float), and pages (integer). Declare variables of type **Book** called **book1** and **book2**, and initialize their members with values of your choice. Display values of structure members of variables **book1** and **book2**.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 13 - Task 3: Accessing Structure Elements (Book)
 * Description: Declare a Book structure with title, author, price, and pages.
 *              Declare two variables, initialize them, and display their values.
 */

#include <iostream>
#include <string>      // Required for string class
using namespace std;

// Declare a structure called 'Book' with the required members
struct Book
{
    string title;      // Member: title (string)
    string author;     // Member: author (string)
    float price;       // Member: price (float)
    int pages;         // Member: pages (integer)
};

int main()
{
    // Declare variables of type Book: book1 and book2
    Book book1, book2;

    // Initialize members of book1
    book1.title = "C++ Programming";
    book1.author = "Bjarne Stroustrup";
    book1.price = 45.99;
    book1.pages = 350;

    // Initialize members of book2
    book2.title = "Data Structures";
    book2.author = "Mark Allen Weiss";
    book2.price = 39.50;
    book2.pages = 280;

    // Display values of structure members of variable book1
    cout << "=== Book 1 Information ===" << endl;
    cout << "Title:  " << book1.title << endl;
    cout << "Author: " << book1.author << endl;
    cout << "Price:  $" << book1.price << endl;
```

```
        cout << "Pages:  " << book1.pages << endl;

    // Display values of structure members of variable book2
    cout << "\n=== Book 2 Information ===" << endl;
    cout << "Title:  " << book2.title << endl;
    cout << "Author: " << book2.author << endl;
    cout << "Price:  $" << book2.price << endl;
    cout << "Pages:  " << book2.pages << endl;

    return 0;
}
```

Expected Output:

```
=== Book 1 Information ===
Title:  C++ Programming
Author: Bjarne Stroustrup
Price:  $45.99
Pages:  350

=== Book 2 Information ===
Title:  Data Structures
Author: Mark Allen Weiss
Price:  $39.50
Pages:  280
```

Explanation: The program declares a **Book** structure with four members: **title** (string), **author** (string), **price** (float), and **pages** (int). Two variables **book1** and **book2** are declared and initialized with different values. This demonstrates that multiple variables can be created from the same structure type, each holding independent data. It also shows the use of floating-point types for monetary values in structures.

4. Conclusion

In this lab session, we successfully explored the concept of structures in C++. We learned that structures are user-defined data types that allow grouping of related data items of different types under a single name. The lab tasks demonstrated practical applications of structure concepts, including:

- Declaring structures with multiple members of different data types (string, int, char, float)
- Initializing structure variables using the dot operator (.)
- Accessing and displaying individual structure member values
- Creating and managing multiple variables of the same structure type

Structures are fundamental to organized data management in C++. They form the basis for more advanced concepts such as nested structures, arrays of structures, and structures in file handling. Understanding structures is essential for developing real-world applications that require complex data representation, such as student records, employee databases, and inventory management systems.

5. References

- Deitel, P. & Deitel, H. (2017). *C++ How to Program* (10th ed.). Pearson.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- UET Peshawar, EE-170L Computer Programming Lab Manual, Spring 2026.